科研训练三

3 研究推进

晏潇 博士
yanxiaosunny@whu.edu.cn

本讲内容

- 上一讲：如何调研论文，提炼研究问题
- 这一讲：如何解决研究问题，形成成果
- 流程：构思，分析，实现，实验，反思



回顾：研究问题从哪来

- 增量改进类：基于 SOTA 方法，提升正确率、效率，降低开销
- 场景迁移类：批处理改流式，静态改支持更新， CPU 换 GPU
- 提出新问题类：抽象共性问题，换一种建模，或重新制定性能度量
  - 发掘一个领域的盲区，通常需要前两类工作进行积累和铺垫



构思解法

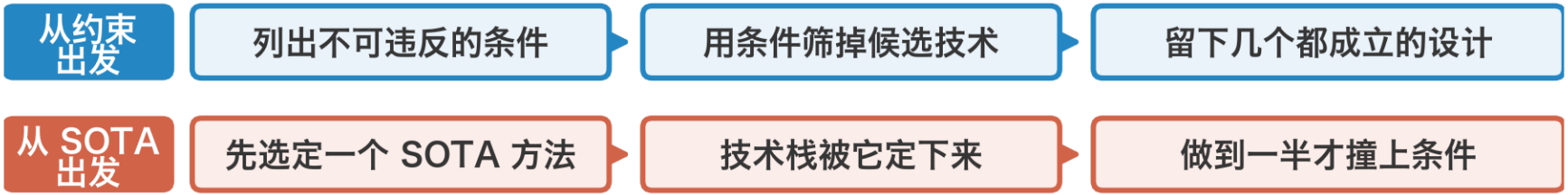
从机理出发构思解法

- 如何理清思路和解法?
  - 把解法里的技术名词全部划掉
  - 能说清「哪里不好、为什么不好、要改成什么」，那才真正找到了问题和解法
  - 从一些实例或实验结果出发是好思路：从直观到抽象



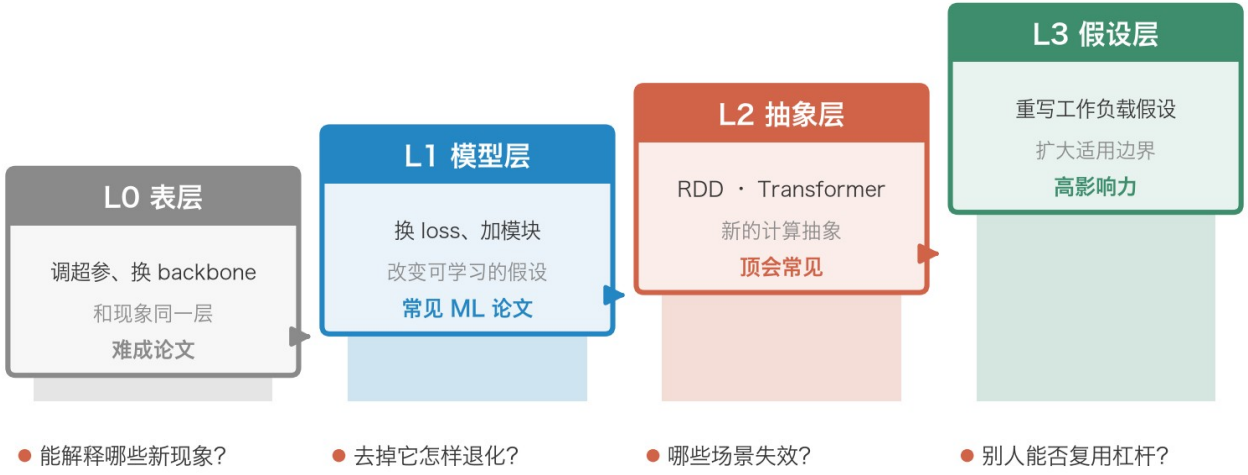
第一性原理：从基本约束和条件出发

- 先问哪些条件不可违反，再挑切入点和设计解决方案
  - 资源：时间、内存、带宽、算力、数据、人力
  - 约束：延迟上限、硬件、隐私、正确性、预算
  - 不变量：必须保持的语义、接口、安全属性
  - 必要条件：去掉它方案就不成立的那个条件
- 从约束出发，产出的是若干仍然成立的设计选择，保守但踩坑少



决定解法的切入点

- 干预点在哪个层次，决定了贡献有多大，越基础的假设越重要
- 往下走一层，通常要换掉一个默认假设
  - L2 换的是数据与计算的抽象，L3 换的是工作负载本身的假设



RDD、Transformer 都属 L2；重写负载假设（如假设可异步）属 L3。

分析解法

Talk is cheap, show me the code

- 产生想法不难，难的是判断它值不值得做，有没有增益
- 光靠思维不行
  - 论文里「我们的方法更好」看着天衣无缝，因为反例没写进去
  - 顺着论文逻辑想下去会觉得已经想通了，动手才发现关键那步走不通
- 要动手实现几个有代表性的 baseline
  - 目的不是比高下，是拿到一手经验：这个任务到底难在哪
  - 复现时遇到的报错、调不上去的指标、反直觉的曲线，才是真材料
- 根据实现中的一手经验，反过来修正对机理的判断
- 通过分析和实验，才能确定想法的可行性

把思路写成文档

- 想清楚才能写清楚，写清楚反过来帮你想清楚
- 文档要回答三个问题
- 短文档， 5 页以内
- 写完之后讲给别人听

思路文档

① 我思路的核心是什么

② 创新点有几个，分别是什么

③ 怎样用最低的代价验证可行性

一句话答不上第 ① 问，就是还没想清楚

讲给别人听

- 发给同门或合作者，请他们提问
- 别人卡住的地方
- 就是文档没写清的地方

Booth et al., The Craft of Research, Ch. 4（写作即思考）。

论证比代码先行

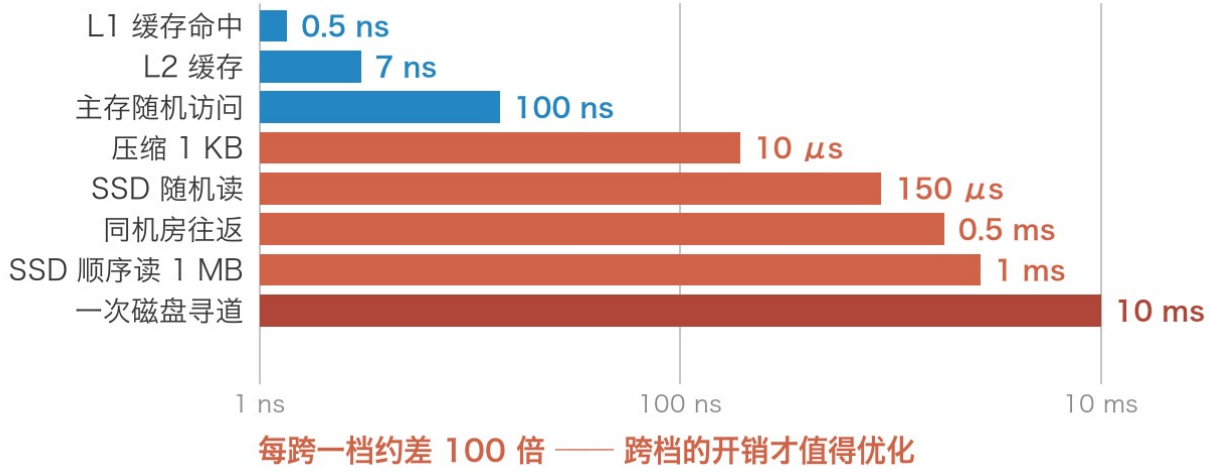
- **先算一遍，还是先写代码**
  - 内存够不够：估算半小时 vs 实现到一半才发现要重写，一学期
  - 瓶颈在 I/O 还是计算：分解耗时几分钟 vs 跑完三轮实验才看出来，三周
  - 理论上能不能到 1 ms：一个公式 vs 上线之后才知道做不到
- **估算不求精确，只求在动手前知道这条路值不值得走**

未知量	先估算	直接上手
内存够不够	一行估算，半小时	实现到一半才发现要重写，一学期
瓶颈在 I/O 还是计算	分解耗时，几分钟	跑完三轮实验才看出来，三周
能不能到 1ms	一个公式	上线之后才知道做不到

Fermi estimation ； Jain, The Art of Computer Systems Performance Analysis, 1991 。

例子： Back-of-envelope Calculation

- **Jeff Dean: Numbers Everyone Should Know**
- **问题：假设大多数数据访问走 SSD，优化内存的随机访问，有意义吗?**



Jeff Dean, Numbers Everyone Should Know, Stanford CS295 talk, 2010 。

算笔账：加速十倍只快 18%

- 一个任务的总耗时里，计算占 20%，数据搬运和等待占 80%
- 比起只优化计算那 20%，搬运动作更值得动
- 这说明什么
  - 优化之前先量清楚时间去哪了，凭感觉选优化对象，十有八九选错
  - 不少「加速某个算子」的工作端到端收益很小，原因就在这里



Amdahl's Law：*speedup = 1 / ((1-p) + p/s)*，*p=0.2*、*s=10* → *1.22×*。

实现

实现：精简，快速迭代

- **20% 的实现完成 80% 的功能**
  - 可以省略：完整 UI 、云端部署、边界情况优化、跨平台支持
  - 不能省略：核心机制跑通、公平 baseline 、正确性检查、关键指标的多次运行
- **单变量更新**
  - 同时改多个变量，说不清是哪个起了作用，实验也没法复现
  - 失败同样有价值：没跑通说明假设错了、边界没考虑到，或实现有 bug
- **确定实现和试验的目标和优先级**
  - 在什么数据集和场景下，对比哪些基线
  - 取得什么样的结果，就基本能确定方法可行
  - 不同领域甚至小问题，能够发表的标准不一样，要熟悉领域
- **每次迭代记下：改了什么、看到什么、下一步做什么**

MVP 的判据是「核心机制能不能跑通」，不是「功能齐不齐」。

不要有工程洁癖，但也不能不严谨

- **两件事要分开看**
  - 代码可以糙：命名、目录结构、异常处理，先能跑通就行
  - 结论不能糙：基线调参强度、测量方法、随机种子，都要规范
- **两种常见误解**
  - 工程洁癖：花两周搭框架、写配置、做可视化，核心机制还没验证
  - 反过来随手跑一次就报数字，被追问时自己都说不清那次是怎么跑的
- **思路定型之后，再全面严谨化**
  - 补边界、加测试、清理代码；在那之前追求整洁，是在给一个可能被推翻的想法做装修

实验设计

实验设计①：全面

- **覆盖代表性的数据集**
  - 挑领域公认、大小合适的；自造先质疑数据
- **覆盖代表性的基线**
  - 思路上有代表性的：说明你比的是哪条路线
  - 近期最强的：说明你比得过当下

覆盖代表性的数据集

- 领域公认：同行都在用的那一两个
- 大小合适：跑得完，且能看出趋势
- 自造数据要先说明为什么要自造

覆盖代表性的基线

- 思路上有代表性：说明你比的是哪条路线
- 近期最强：说明你比得过当下
- 调参强度对齐，否则比的是调参水平

数据集和基线都要能说出「为什么是它」

实验设计②：科学

- **控制变量**
  - 对基线的参数和实现做同等强度的调优，每次只改一个变量
- **多次重复取平均**
  - 每个数字至少重复 3–5 次，报告均值和方差
- **只看 p 值不够，还要说清差了多少**



Demšar, JMLR 2006 ； Montgomery, Design and Analysis of Experiments, 9th ed 。

实验设计③：明确

- **实验配置要写清**
  - 硬件、软件栈、数据集版本、随机种子
- **测试指标要写清**
  - 指标怎么定义、越大越好还是越小越好、报告的是哪个统计量
- **别人照着你写的东西能不能复现，是重要检验标准**

实验配置要写清

- 硬件：CPU/GPU 型号、内存
- 软件栈：框架与版本、编译选项
- 数据集版本与切分方式
- 随机种子

测试指标要写清

- 怎么定义：公式或口径
- 方向：越大越好还是越小越好
- 报哪个统计量：均值 / 中位数 / 分位
- 测多少次

别人照着你写的东西能不能复现，是唯一的检验标准

实验设计④：完整

- 需要论证的点都要有对应实验
  - 已有方法的弱点，要有实验说明它真的弱
  - 我们的优势，要有实验说明它来自哪个机制
- 只报好的那一半，评审会自己去补另一半

Claim（主张）	证据类型	关键指标
方案降低迭代延迟	端到端 + 分阶段耗时	总时间 · 各阶段时间
收益来自减少 I/O	I/O 量测量 + 机制消融	I/O 字节数 · 命中率
额外开销可接受	内存 / 恢复 / 管理开销	内存占用 · 恢复时间

每个 Claim 至少一组直接证据；每个关键机制至少一次消融

细节实验：端到端之外还要展示什么

- 只报端到端数字通常不够，要能够展示一些有趣的现象
- 几类值得单独做的实验
  - 每一类都要能说清它回答的是哪个质疑
- 有的时候写我们在什么情况下不好，反而更容易让人相信那些好的数字



列出实验计划

- **写出来才能想清楚**
  - 数据集、基线、配置，具体是哪几个、参数从哪来
  - 明确主次关系和优先级，时间不够时先砍次要的
- **排优先级的一个办法**
  - 先跑最可能推翻自己结论的那个实验

实验	数据集	基线	优先级	预计耗时
主实验：端到端对比	数据集 A+B	基线 1/2/3	最高	3 天
消融：去掉核心机制	数据集 A	自身变体	高	1 天
敏感性：主参数扫描	数据集 A	—	中	半天

先跑最可能推翻自己结论的那个实验

应对失败与总结反思

应对失败

- 解决问题的尝试，并不总是成功
  - 大多数想法最后没做成，这不是例外，是常态
- 更难受的情况：前路不明
  - 不是失败了，而是不知道下一步往哪走，也不确定还有没有改善空间
- 这时候该做的是停下编程
  - 留出时间复盘：哪些现象解释不通，原始假设哪一条可能错了
  - 给导师或同学讲一遍，别人常常一句话指出你绕了远路
  - 确认这条路走不通就换问题，沉没成本不该决定研究方向

分析和反思

- **收敛性、复杂度、资源开销，跟已有工作比是什么位置**
  - 复杂度是降了一阶，还是常数项小一点？这决定结论能写多大
- **场景适应性和易用性**
  - 换个规模、换份数据还成立吗
- **方法有什么缺项和改进方向**
  - 把没做到的一条条列出来，说明是没时间做还是这样做不了
- **能说清缺陷的人，比只讲优点的人更可信**
  - 评审不会因为你有局限而拒稿，没有完美的没有缺陷的方法，能够说清楚缺陷和未来方向也非常重要

常见的逻辑谬误

- **四种最常见**
  - 相关当因果：有没有控制混淆变量
  - 以偏概全：一个数据集说明不了全部
  - 选择性汇报：只展示有利的 5 次，藏起失败的 95 次
  - 不公平比较：新方法调一周， baseline 跑默认参数
- **四种更隐蔽**
  - 指标偷换：优化了离线 AUC ，线上指标一动不动
  - 混杂变量：一次改 N 个变量，功劳全归新算法
  - 事后解释：先看结果再编假设和解释
  - 缺少反证：实验全设计成「能成功」的样子
- **交稿前自己反问五轮：这个结论在反问下还站得住吗?**

Nuzzo, Nature 2014 ； Kerr 1998 ； Nosek et al., Science 2012 ； Popper, 1959 。

杂谈

- **工作速度越来越快，热点变换越来越频繁**
  - 善用模型和自动化工具
- **劳逸结合**
  - 高效工作和思考的时间才是有效时间，不要盲目堆时长
  - 保持专注，特定时间不允许打扰
  - 规律运动和放松，保持好状态
- **规律汇报和研讨**
  - 2-3 周应该和导师或者同学有一次深度讨论
  - 整个项目周期有 3-4 次讨论
  - 通过准备讨论，理清思路和进行反思

Nuzzo, Nature 2014 ； Kerr 1998 ； Nosek et al., Science 2012 ； Popper, 1959 。

准备研究成果汇报

- **定义清楚问题和场景**
  - 听众没读过你读的东西，前三页讲不清问题，后面全白讲
- **介绍相关工作的主要思路和局限性**
  - 讲思路和局限，不要念论文；指向你准备补的那个洞
- **介绍自己的工作，聚焦 2–3 个主要的点**
  - 别按时间顺序流水账，挑最能说明「为什么有效」的点
  - 理清每个子步骤的输入和输出，听众才跟得上
- **展示实验结果，明确实验结论**
  - 每张图表配一句话说明它证明了什么